Dev Day ∼42: Revised State Notation

State Machine with Physics-Informed Bookkeeping

VSEPR-SIM Development

Version 1.0.0 — Revised Notation Draft

Contents

1 Motivation: Why the Old Notation Had to Go **2**

2 State Declaration (Initial Conditions) **2**
- 2.1 Formal Definition . . . 2
- 2.2 Design Rationale . . . 2
- 2.3 What S_0 Does Not Contain . . . 3

3 Outcome State **3**
- 3.1 Formal Definition . . . 3
- 3.2 Structural State (Φ) . . . 3
- 3.3 Event Log (Ω) . . . 4
- 3.4 Quality Flags (Q) . . . 4

4 Transformation Layer **4**
- 4.1 The Only Thing That Actually Matters . . . 4
- 4.2 Model Fidelity (Λ) . . . 5
- 4.3 Composability . . . 5

5 Mass–Energy Sanity Constraint **5**
- 5.1 Don't Let It Lie to You . . . 5
- 5.2 Implementation . . . 6

6 What You Actually Built **6**
- 6.1 State Machine Diagram . . . 7

7 C++ Implementation **7**
- 7.1 StateC: The Initial State Container . . . 7
- 7.2 The Transformation Operator . . . 7
- 7.3 Composition Enum . . . 7
- 7.4 ModelLevel Enum (Λ) . . . 8
- 7.5 EnergyBalance: The Honesty Ledger . . . 8

8 Integration with Existing Codebase **8**
- 8.1 Relationship to atomistic::State . . . 8
- 8.2 Relationship to SimulationState . . . 9
- 8.3 Relationship to CGSystemState . . . 9

9 Multi-Fidelity Workflow Example **9**

10 Relationship to Deep Research Engine **9**

11 Project Tree Context **10**

12 Symbol Reference **10**

13 Summary **11**

1 Motivation: Why the Old Notation Had to Go

The symbol I collides with literally every field in physics:

- Electric current ($I = Q/t$)
- Moment of inertia ($I = mr^2$)
- Identity matrix (I_n)
- Intensity ($I = P/A$)
- Action integral ($I = \int L\,dt$)
- Ionization energy ($I_1, I_2, \ldots$)
- Nuclear spin quantum number

Using I as the initial-state symbol guaranteed confusion in any document that also discusses electrostatics, rotational mechanics, matrix algebra, or spectroscopy.

The fix is simple: use a symbol that does not collide.

$$\boxed{S_0 = \{m_0,\ E_0,\ x_0,\ v_0,\ C,\ \mathcal{E},\ K,\ \Sigma\}} \tag{1}$$

S_0 is clean, neutral, and non-annoying.

2 State Declaration (Initial Conditions)

2.1 Formal Definition

The initial state container is:

$$S_0 = \{m_0,\ E_0,\ x_0,\ v_0,\ C,\ \mathcal{E},\ K,\ \Sigma\} \tag{2}$$

Symbol	Name	Description
S_0	Initial state container	Clean, neutral, non-annoying
m_0	Initial mass	Total system mass (amu)
E_0	Initial energy	Total system energy (kcal/mol)
x_0	Spatial configuration	Center-of-mass position (Å)
v_0	Velocity / motion state	Center-of-mass velocity (Å/ps)
C	Composition	Atomistic / bead / hybrid / macro
$\mathcal{E}$	Environment	T, P, fields, radiation, flow
K	Constraints	Geometry, bonding rules, allowed transitions
Σ	Source / provenance tags	Where did this state come from?

2.2 Design Rationale

Each field has a clear physical or bookkeeping purpose:

- m_0, E_0: Scalar aggregates. Even if the underlying state has N particles with individual masses, the container-level mass and energy are always well-defined.

- x_0, v_0: The center-of-mass phase-space point. Individual particle coordinates live in the detailed `State` object that S_0 wraps.
- C: A bookkeeping tag, not a physics quantity. Tells the engine which force evaluators and integrators are valid.
- $\mathcal{E}$: External conditions that the system responds to but does not generate. Temperature, pressure, electric/magnetic fields, radiation flux, shear rate.
- K: Hard constraints that the engine must respect. Frozen atoms, bonding rules, symmetry groups.
- Σ: Provenance chain. If you cannot answer "where did this number come from?" then your simulation is not reproducible.

2.3 What S_0 Does Not Contain

S_0 intentionally omits:

- Force vectors (these are computed, not stored)
- Scratch buffers (temporary workspace for integrators)
- Visualization state (rendering is a separate layer)
- File I/O handles (state is pure data)

3 Outcome State

3.1 Formal Definition

$$S_f = \{m_f,\ E_f,\ \Phi,\ \mathcal{S},\ \Pi,\ \Omega,\ Q\} \tag{3}$$

Symbol	Name	Description
S_f	Final state	The result of transformation
m_f	Final mass	Total system mass after evolution
E_f	Final energy	Total system energy after evolution
Φ	Structural state	Geometry/topology class
$\mathcal{S}$	Stability metric	Thermodynamic and kinetic stability quantifier
Π	Performance properties	Transport / material properties (diffusion, viscosity, etc.)
Ω	Event log	Reactions, transitions, failures during T
Q	Quality / confidence flags	How much should you trust this result?

3.2 Structural State (Φ)

Φ classifies the geometry and topology of the outcome:

- Geometry class: linear, trigonal planar, tetrahedral, octahedral, ...
- Topology: chain, ring, branched, lattice, amorphous

- Symmetry score: 0 (no symmetry) to 1 (perfect)
- Coordination number (average)

3.3 Event Log (Ω)

Ω records every discrete event during transformation:

- Bond breaking / forming events
- Phase transitions
- Constraint violations (flagged, not silenced)
- Energy drift warnings
- Failure modes

Each event carries: step number, type tag, human-readable description, and associated energy change ΔE.

3.4 Quality Flags (Q)

Q is the honesty layer. It tells you:

- Was energy conserved?
- Was mass conserved?
- Did forces converge?
- Were symmetry constraints respected?
- What is the absolute energy drift?

If Q reports `energy_conserved = false`, the result is suspect and the user is informed. The engine does NOT silently continue with broken bookkeeping.

4 Transformation Layer

4.1 The Only Thing That Actually Matters

$$\boxed{S_f = T(S_0,\ t,\ \Lambda)} \tag{4}$$

Symbol	Name	Description
T	Transformation operator	Your engine
t	Time / iteration domain	How long to run
Λ	Model fidelity level	C-level here, not physics-resolved

This is the whole point:

You are no longer solving reality. You are mapping states through controlled abstraction.

4.2 Model Fidelity (Λ)

The Λ parameter tells the engine *how much physics* to include:

Level	Name	Description
Λ_0	C-Empirical	Classical empirical: harmonic bonds, LJ, Coulomb
Λ_1	C-Reactive	Classical with bond-order potentials (ReaxFF-like)
Λ_2	C-Polarizable	Classical with induced dipoles / SCF polarization
Λ_3	CG-Reduced	Coarse-grained reduced model
Λ_4	CG-Enriched	Coarse-grained enriched descriptor model
Λ_5	Hybrid-AdRes	Adaptive resolution (atomistic core + CG shell)

At C-level (Λ_0 through Λ_2), we are not resolving quantum mechanics or electronic structure. We are mapping states through controlled classical approximation. The fidelity level is an explicit parameter, not a hidden assumption.

4.3 Composability

Because T maps $S_0 \to S_f$, and S_f can be wrapped back into an S_0 (with updated provenance), transformations compose:

$$S_f^{(3)} = T\big(T\big(T(S_0,\, t_1,\, \Lambda_a),\, t_2,\, \Lambda_b\big),\, t_3,\, \Lambda_c\big) \tag{5}$$

This enables multi-fidelity workflows: optimize at Λ_0 (cheap), refine at Λ_2 (expensive), coarsen to Λ_3 (fast screening).

5 Mass–Energy Sanity Constraint

5.1 Don't Let It Lie to You

Even at C-level, enforce:

$$E_0 + E_{\text{in}} - E_{\text{out}} \approx E_f + E_{\text{loss}} \tag{6}$$

$$m_0 + m_{\text{in}} - m_{\text{out}} \approx m_f \tag{7}$$

Term	Name	Description
E_0	Initial energy	From S_0
E_{in}	Injected energy	Thermostat, external fields, radiation
E_{out}	Removed energy	Friction, thermostat coupling, damping
E_f	Final energy	From S_f
E_{loss}	Dissipative loss	Explicitly tracked, never hidden
m_0	Initial mass	From S_0
m_{in}	Added mass	Particle creation events
m_{out}	Removed mass	Particle destruction events
m_f	Final mass	From S_f

If this drifts, your "approximation layer" quietly becomes fiction.

5.2 Implementation

The `EnergyBalance` struct tracks all six terms. At every step:

1. Compute $\delta E = |(E_0 + E_{\text{in}} - E_{\text{out}}) - (E_f + E_{\text{loss}})|$
2. If $\delta E > \varepsilon_E$: record a warning in Ω, set Q.`energy_conserved` = false
3. Compute $\delta m = |(m_0 + m_{\text{in}} - m_{\text{out}}) - m_f|$
4. If $\delta m > \varepsilon_m$: same treatment

The tolerances ε_E and ε_m are explicit parameters in `TransformConfig`. They are not hidden. They are not "close enough." They are the contract between the engine and the user.

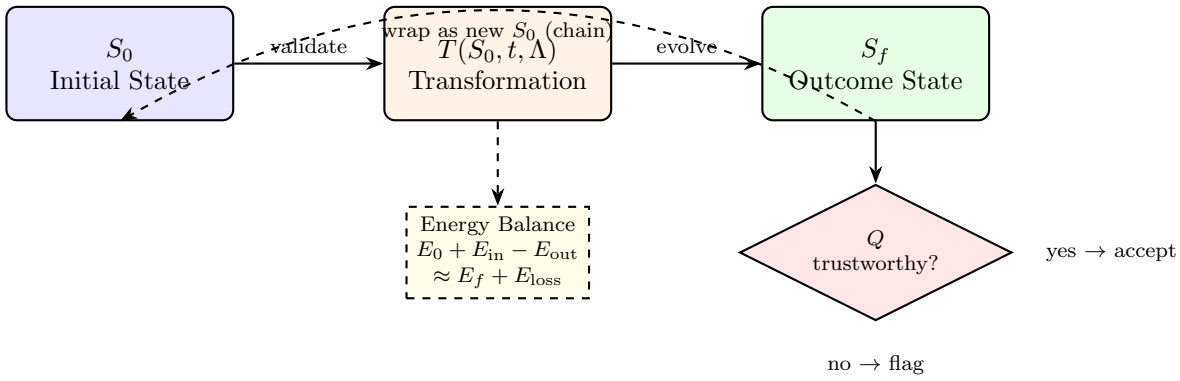
6 What You Actually Built

Whether you admit it or not, this system is:

1. **A state machine** — $S_0 \to S_f$ with well-defined transitions
2. **With physics-informed bookkeeping** — mass-energy balance, provenance, quality flags
3. **Feeding a multi-scale transformation engine** — Λ selects fidelity from atomistic through coarse-grained to hybrid
4. **With traceable data lineage** — Σ provenance tags chain through every transformation

Which is exactly what serious simulation frameworks do, except they pretend it is more mystical.

6.1 State Machine Diagram



7 C++ Implementation

7.1 StateC: The Initial State Container

Listing 1: StateC — Dev Day ∼42 initial state (atomistic/core/state_c.hpp)

```
struct StateC {
    double mass;           // m_0
    double energy;         // E_0
    Vec3   position;       // x_0
    Vec3   velocity;       // v_0

    Composition   comp;         // C
    Environment   env;          // E (calligraphic)
    Constraints   constraints;  // K
    SourceTag     provenance;   // Sigma
};
```

7.2 The Transformation Operator

Listing 2: evolve — The engine becomes a function

```
StateC evolve(const StateC& s0, double dt, ModelLevel level);
```

And the full-output version:

Listing 3: Full evolve with OutcomeState

```
OutcomeState evolve(const StateC& s0, const TransformConfig& config);
```

7.3 Composition Enum

```
enum class Composition : uint8_t {
    Atomistic = 0,   // Full atomic resolution
    Bead      = 1,   // Coarse-grained bead
    Hybrid    = 2,   // Mixed atomistic + bead
    Macro     = 3    // Continuum / macro-scale
};
```

7.4 ModelLevel Enum (Λ)

```
enum class ModelLevel : uint8_t {
    C_Empirical   = 0,  // Harmonic bonds, LJ, Coulomb
    C_Reactive    = 1,  // Bond-order potentials
    C_Polarizable = 2,  // Induced dipoles / SCF
    CG_Reduced    = 3,  // Coarse-grained reduced
    CG_Enriched   = 4,  // CG enriched descriptor
    Hybrid_AdRes  = 5   // Adaptive resolution
};
```

7.5 EnergyBalance: The Honesty Ledger

Listing 4: EnergyBalance — mass-energy sanity constraint

```
struct EnergyBalance {
    double E_0{}, E_in{}, E_out{}, E_f{}, E_loss{};
    double m_0{}, m_in{}, m_out{}, m_f{};

    double energy_drift() const {
        return std::abs((E_0 + E_in - E_out) - (E_f + E_loss));
    }
    double mass_drift() const {
        return std::abs((m_0 + m_in - m_out) - m_f);
    }
    bool energy_sane(double tol = 1e-6) const {
        return energy_drift() < tol;
    }
    bool mass_sane(double tol = 1e-10) const {
        return mass_drift() < tol;
    }
};
```

8 Integration with Existing Codebase

8.1 Relationship to atomistic::State

The existing `atomistic::State` struct (defined in `atomistic/core/state.hpp`) is the detailed particle-level state:

$$\texttt{State} = (N,\ X,\ V,\ T,\ Q,\ M,\ \texttt{type},\ B,\ L,\ F,\ E,\ \texttt{box}) \tag{8}$$

`StateC` wraps this with:

- Container-level aggregates (m_0, E_0, x_0, v_0)
- Bookkeeping metadata (C, $\mathcal{E}$, K, Σ)
- A `const State*` pointer for zero-copy access to the detailed data

The factory method:

```
StateC sc = StateC::from_state(full_state, provenance_tag);
```

computes the COM position/velocity and total mass/energy from the full particle data.

8.2 Relationship to SimulationState

The existing `vsepr::SimulationState` (in `src/sim/sim_state.hpp`) is the engine-side state machine that owns the molecule, coordinates, velocities, forces, and integrator state. It corresponds to the *interior* of the transformation operator T.

In the revised notation:

- `StateC` = the *contract* at the boundary (what goes in, what comes out)
- `SimulationState` = the *mechanism* inside T (how integration happens)

8.3 Relationship to CGSystemState

The existing `vsepr::cli::CGSystemState` (in `include/cli/system_state.hpp`) holds beads, environment states, and scene metadata. In the revised notation, this is a specialized S_0 with $C = \text{Bead}$.

The `Composition` enum makes this explicit: when $C = \text{Bead}$, the engine knows to use coarse-grained force evaluators and CG integrators.

9 Multi-Fidelity Workflow Example

1. **Parse input**: Load `water_cluster.xyz` into `atomistic::State`
2. **Wrap**: `StateC s0 = StateC::from_state(state, {.origin="file:water_cluster.xyz"})`
3. **Optimize** (cheap): `auto s1 = evolve(s0, 0.1, ModelLevel::C_Empirical)`
4. **Refine** (expensive): `auto s2 = evolve(s1, 0.001, ModelLevel::C_Polarizable)`
5. **Coarsen**: Map atomistic result to beads, set $C = \text{Bead}$
6. **Screen** (fast): `auto s3 = evolve(s2_cg, 1.0, ModelLevel::CG_Reduced)`
7. **Inspect**: Check Q flags at each stage. If any report `energy_conserved = false`, the pipeline stops.

Each step carries its provenance in Σ. The chain is:

$$\Sigma : \texttt{file:water\_cluster.xyz} \to \texttt{evolve}(\Lambda_0) \to \texttt{evolve}(\Lambda_2) \to \texttt{coarsen} \to \texttt{evolve}(\Lambda_3) \tag{9}$$

10 Relationship to Deep Research Engine

The Deep Research Engine (`atomistic/research/deep_research.hpp`) implements a 4-phase pipeline:

Phase	Name	C	Λ
10A	Atomistic Enumeration	Atomistic	Λ_0 (C-Empirical)
20B	Bead Formation	Bead	Λ_3 (CG-Reduced)
31C	MD Assembly	Bead	Λ_3 (CG-Reduced)
41A	Lattice Assembly	Macro	— (blank call)

In the revised notation, each phase is a transformation:

$$S_f^{(10A)} = T(S_0^{\text{seed}},\ t_{10A},\ \Lambda_0) \tag{10}$$

$$S_f^{(20B)} = T(S_f^{(10A)} \to \text{Bead},\ t_{20B},\ \Lambda_3) \tag{11}$$

$$S_f^{(31C)} = T(S_f^{(20B)},\ t_{31C},\ \Lambda_3) \tag{12}$$

$$S_f^{(41A)} = T(S_f^{(31C)} \to \text{Macro},\ 0,\ \text{infer}) \tag{13}$$

The SHA-512 hash system in the deep research engine maps directly to the Σ provenance tags: each pathway fingerprint becomes a `SourceTag.hash`.

11 Project Tree Context

```
atomistic/
  core/
    state.hpp           <-- Particle-level state (N, X, V, M, ...)
    state_c.hpp         <-- Dev Day ~42 revised notation (THIS FILE)
    linalg.hpp
  integrators/
    fire.hpp            <-- FIRE optimizer (used inside T)
  research/
    deep_research.hpp   <-- 4-phase pipeline (uses StateC notation)
  reaction/
    discovery.hpp       <-- Discovery engine (reaction mining)

coarse_grain/
  core/
    bead.hpp            <-- Bead struct (C = Bead representation)
    environment_state.hpp <-- Environment-responsive state (X_B)
  models/
    model_selector.hpp  <-- ModelTier (maps to Lambda levels)

src/sim/
  sim_state.hpp         <-- SimulationState (interior of T)
  optimizer.hpp         <-- FIRE parameters

include/cli/
  system_state.hpp      <-- CGSystemState (C = Bead specialization)

docs/
  section2_state_ontology.tex          <-- Original state ontology
  section_state_c_revised_notation.tex <-- THIS DOCUMENT
```

12 Symbol Reference

Complete symbol table for the revised notation:

Symbol	LaTeX	C++ Type	Meaning
S_0	`S_0`	`StateC`	Initial state container
S_f	`S_f`	`OutcomeState`	Outcome state
T	`T`	`evolve()`	Transformation operator
t	`t`	`TransformConfig`	Time / iteration domain
Λ	`\Lambda`	`ModelLevel`	Model fidelity level
m_0	`m_0`	`double mass`	Initial total mass
E_0	`E_0`	`double energy`	Initial total energy
x_0	`x_0`	`Vec3 position`	COM position
v_0	`v_0`	`Vec3 velocity`	COM velocity
C	`C`	`Composition`	Representation level
$\mathcal{E}$	`\mathcal{E}`	`Environment`	External conditions
K	`K`	`Constraints`	Rules / restrictions
Σ	`\Sigma`	`SourceTag`	Provenance tags
Φ	`\Phi`	`StructuralState`	Geometry/topology class
$\mathcal{S}$	`\mathcal{S}`	`StabilityMetric`	Stability quantifier
Π	`\Pi`	`PerformanceProperties`	Transport properties
Ω	`\Omega`	`EventLog`	Event log
Q	`Q`	`QualityFlags`	Confidence flags

13 Summary

Dev Day ∼42 establishes the revised notation for the VSEPR-SIM state machine:

1. S_0 **replaces** I as the initial state symbol. No more collisions with current, inertia, identity, or intensity.
2. $S_f = T(S_0, t, \Lambda)$ is the only equation that matters. Everything else is bookkeeping to make T honest.
3. **Mass-energy sanity** is enforced at every step, not assumed. If it drifts, Q flags it.
4. **Provenance** (Σ) chains through every transformation, making the entire pipeline reproducible and inspectable.
5. **Composition** (C) and **fidelity** (Λ) are explicit parameters, not hidden assumptions.

The C++ implementation lives in `atomistic/core/state_c.hpp` and integrates with the existing `atomistic::State`, `SimulationState`, and `CGSystemState` without breaking any existing interfaces.